

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Struktur Data Single Linked List pada Java”

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

KATA PENGANTAR

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Doubly Linked List adalah struktur data berupa rangkaian node yang saling terhubung, di mana setiap node menyimpan tiga hal: data, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev). Berbeda dengan singly linked list yang hanya bisa traversal satu arah, doubly linked list memungkinkan traversal dua arah — maju maupun mundur.

Keunggulan utamanya adalah fleksibilitas dalam operasi seperti insert dan delete, karena tidak perlu melacak node sebelumnya secara manual. Ini terlihat jelas pada kode di atas, di mana class Lagu memiliki field `next_2021` dan `prev_2021` untuk merepresentasikan hubungan dua arah antar lagu — cocok untuk fitur seperti playlist musik yang bisa diputar maju dan mundur.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Node pada struktur data Doubly Linked List.
2. Mengimplementasikan operasi penambahan data (Insertion) di berbagai posisi, dan penghapusan di berbagai posisi
3. Menerapkan operasi penelusuran secara dua arah, baik dari head menuju tail, maupun tail menuju head

BAB 2 PEMBAHASAN

3.1 Program NodeDLL

```
package org.example;

import java.util.*;

public class NodeDLL_2511532021{

    int data_2021;

    NodeDLL_2511532021 next_2021;

    NodeDLL_2511532021 prev_2021;

    public NodeDLL_2511532021(int data_2021){

        this.data_2021 = data_2021;

        this.next_2021 = null;

        this.prev_2021 = null;

    }

}
```

Class NodeDLL_2511532021 adalah representasi dari sebuah node dalam Doubly Linked List, yang menyimpan tiga komponen: data_2021 bertipe int untuk menyimpan nilai/data integer, next_2021 bertipe NodeDLL_2511532021 sebagai pointer ke node berikutnya, dan prev_2021 bertipe NodeDLL_2511532021 sebagai pointer ke node sebelumnya. Konstruktornya menerima satu parameter data_2021 untuk menginisialisasi nilai data, sementara next_2021 dan prev_2021 diset null karena node baru belum terhubung ke node manapun.

3.2 Program InsertDLL

Program Java tersebut mengimplementasikan operasi penambahan node pada

Single Linked List (SLL). Class `TambahSSL_2511532021` memiliki method untuk menambah node di depan (`insertAtFront_2021()`), di belakang (`insertAtEnd_2021()`), dan pada posisi tertentu (`insertPos_2021()`). Selain itu, terdapat method `printList_2021()` untuk menampilkan isi linked list. Pada method `main`, program membuat linked list awal lalu melakukan beberapa penambahan node untuk menunjukkan cara kerja manipulasi data secara dinamis pada linked list.

```
package org.example;
```

```
public class InsertDLL_2511532021 {
```

```
    public static NodeDLL_2511532021 InsertBegin(NodeDLL_2511532021
head_2021, int data_2021) {
```

```
        NodeDLL_2511532021 new_node_2021 = new
NodeDLL_2511532021(data_2021);
```

```
        new_node_2021.next_2021 = head_2021;
```

```
        if (head_2021 != null) {
```

```
            head_2021.prev_2021 = new_node_2021;
```

```
        }
```

```
        return new_node_2021;
```

```
    }
```

```
    public static NodeDLL_2511532021 InsertEnd(NodeDLL_2511532021
head_2021, int newData_2021) {
```

```
        NodeDLL_2511532021 newNode_2021 = new
NodeDLL_2511532021(newData_2021);
```

```
        if (head_2021 == null) {
```

```
            return newNode_2021;
```

```
        } else {
```

```

NodeDLL_2511532021 curr_2021 = head_2021;

while (curr_2021.next_2021 != null) {

    curr_2021 = curr_2021.next_2021;

}

curr_2021.next_2021 = newNode_2021;

newNode_2021.prev_2021 = curr_2021;

}

return head_2021;

}

```

```

public static NodeDLL_2511532021 insertAtPosition(NodeDLL_2511532021
head_2021, int pos_2021, int new_data_2021){

    NodeDLL_2511532021 new_node_2021 = new
NodeDLL_2511532021(new_data_2021);

```

```

    if (pos_2021 == 1){

        new_node_2021.next_2021 = head_2021;

        if (head_2021 != null){

            head_2021.prev_2021 = new_node_2021;

        }

        head_2021 = new_node_2021;

        return head_2021;

    }

```

```

NodeDLL_2511532021 curr_2021 = head_2021;

```

```

    for (int i = 1; i < pos_2021 - 1 && curr_2021 != null; i++){

        curr_2021 = curr_2021.next_2021;

    }

    if (curr_2021 == null){

        System.out.println("Posisi tidak ada");

        return head_2021;

    }

    new_node_2021.prev_2021 = curr_2021;

    new_node_2021.next_2021 = curr_2021.next_2021;

    curr_2021.next_2021 = new_node_2021;

    if(new_node_2021.next_2021 != null){

        new_node_2021.next_2021.prev_2021 = new_node_2021;

    }

    return head_2021;

}

public static void printlist(NodeDLL_2511532021 head_2021){

    NodeDLL_2511532021 curr_2021 = head_2021;

    while (curr_2021 != null){

        System.out.print(curr_2021.data_2021 + " <-> ");

        curr_2021 = curr_2021.next_2021;

    }

    System.out.println();

}

```



```
public static void main(String[] args){

    NodeDLL_2511532021 head_2021 = new NodeDLL_2511532021(2);

    head_2021.next_2021 = new NodeDLL_2511532021(3);

    head_2021.next_2021.prev_2021 = head_2021;

    head_2021.next_2021.next_2021 = new NodeDLL_2511532021(5);

    head_2021.next_2021.next_2021.prev_2021 = head_2021.next_2021;


    System.out.print("DLL Awal: ");

    printlist(head_2021);


    head_2021 = InsertBegin(head_2021,1);

    System.out.print(

        "simpul 1 ditambah di awal: "

    );

    printlist(head_2021);


    System.out.print(

        "simpul 6 ditambah di akhir: "

    );


    int data_2021 = 6;

    head_2021 = InsertEnd(head_2021, data_2021);

    printlist(head_2021);
```

```

        System.out.print("Tambah node 4 di posisi 4: ");

        int data2_2021 = 4;

        int pos_2021 = 4;

        head_2021 = insertAtPosition(head_2021, pos_2021, data2_2021);

        printlist(head_2021);

    }

}

```

Class InsertDLL_2511532021 mengimplementasikan tiga operasi insert pada Doubly Linked List: InsertBegin untuk menambah node di awal dengan menggeser head dan menghubungkan pointer prev-nya, InsertEnd untuk menambah node di akhir dengan traversal hingga node terakhir lalu menyambungkan pointer next dan prev-nya, serta insertAtPosition untuk menyisipkan node di posisi tertentu dengan traversal ke posisi target lalu menghubungkan pointer dua arah di antara node sebelum dan sesudahnya. Pada main, dibuat DLL awal dengan node 2 <-> 3 <-> 5, lalu diuji ketiga operasi insert tersebut secara berurutan sehingga hasil akhirnya menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6.

3.3 Program PenelusuranDLL

Class PenulusuranDLL_2511532021 mengimplementasikan dua operasi traversal pada Doubly Linked List: forwardTraversal_2021 yang menelusuri list dari head ke akhir menggunakan pointer next, dan backwardTraversal_2021 yang menelusuri list dari tail ke awal menggunakan pointer prev. Pada main, dibuat DLL dengan tiga node 1 <-> 2 <-> 3 yang dihubungkan secara manual dua arah, lalu diuji kedua traversal tersebut — forward menghasilkan 1 <-> 2 <-> 3 dan backward menghasilkan 3 <-> 2 <-> 1.

```
package org.example;
```

```
public class PenulusuranDLL_2511532021{

    static void forwardTraversal_2021(NodeDLL_2511532021 head_2021){

        NodeDLL_2511532021 curr_2021 = head_2021;

        while(curr_2021 != null){

            System.out.print(curr_2021.data_2021 + " <-> ");

            curr_2021 = curr_2021.next_2021;

        }

        System.out.println();

    }

    static void backwardTraversal_2021(NodeDLL_2511532021 tail_2021){

        NodeDLL_2511532021 curr_2021 = tail_2021;
```

```
while(curr_2021 != null){

    System.out.print(curr_2021.data_2021 + " <-> ");

    curr_2021 = curr_2021.prev_2021;

}

System.out.println();

}

public static void main(String[] args){

    NodeDLL_2511532021 head_2021 = new NodeDLL_2511532021(1);

    NodeDLL_2511532021 second_2021= new NodeDLL_2511532021(2);

    NodeDLL_2511532021 third_2021= new NodeDLL_2511532021(3);

    head_2021.next_2021 = second_2021;

    second_2021.prev_2021 = head_2021;

    second_2021.next_2021 = third_2021;

    third_2021.prev_2021 = second_2021;

    System.out.println("Penelusuran Maju: ");

    forwardTraversal_2021(head_2021);

    System.out.println("Penelusuran Mundur: ");

    backwardTraversal_2021(third_2021);
```

$\}$ $\}$

3.4 Program HapusDLL

Class HapusDLL_2511532021 mengimplementasikan tiga operasi hapus pada Doubly Linked List: delHead_2021 untuk menghapus node pertama dengan menggeser head ke node berikutnya dan memutus pointer prev-nya, delLast untuk menghapus node terakhir dengan traversal hingga akhir lalu memutus pointer next dari node sebelumnya, serta delPost untuk menghapus node di posisi tertentu dengan traversal ke posisi target lalu menghubungkan ulang pointer next dan prev dari node di kiri dan kanannya. Pada main, dibuat DLL awal 1 <-> 2 <-> 3 <-> 4 <-> 5, lalu diuji ketiga operasi hapus secara berurutan — hapus head menghasilkan 2 <-> 3 <-> 4 <-> 5, hapus tail menghasilkan 2 <-> 3 <-> 4, dan hapus node ke-2 menghasilkan 2 <-> 4.

```
package org.example;
```

```
public class HapusDLL_2511532021{

    public static NodeDLL_2511532021 delHead_2021(NodeDLL_2511532021
head_2021){

        if (head_2021 == null){

            return null;

        }

        NodeDLL_2511532021 temp_2021 = head_2021;

        head_2021 = head_2021.next_2021;

        if (head_2021 != null){

            head_2021.prev_2021 = null;

        }

        return head_2021;

    }
```

```

        public static NodeDLL_2511532021 delLast(NodeDLL_2511532021
head_2021){

    if (head_2021 == null){

        return null;

    }

    if (head_2021.next_2021 == null){

        return null;

    }


    NodeDLL_2511532021 curr_2021 = head_2021;


    while (curr_2021.next_2021 != null){

        curr_2021 = curr_2021.next_2021;

    }


    if(curr_2021.prev_2021 != null){

        curr_2021.prev_2021.next_2021 = null;

    }

    return head_2021;

}


    public static NodeDLL_2511532021 delPost(NodeDLL_2511532021 head_2021,
int pos_2021){

    if(head_2021 == null){

        return head_2021;

```

```
}

NodeDLL_2511532021 curr_2021 = head_2021;

for (int i = 1; curr_2021 != null && i < pos_2021; ++i){
    curr_2021 = curr_2021.next_2021;
}

if(curr_2021 == null){
    return head_2021;
}

if (curr_2021.prev_2021 != null){
    curr_2021.prev_2021.next_2021 = curr_2021.next_2021;
}

if (curr_2021.next_2021 != null){
    curr_2021.next_2021.prev_2021 = curr_2021.prev_2021;
}

if(head_2021 == curr_2021){
    head_2021 = curr_2021.next_2021;
}

return head_2021;
}
```



```

public static void printList(NodeDLL_2511532021 head_2021) {

    NodeDLL_2511532021 curr_2021 = head_2021;

    while(curr_2021 != null){

        System.out.print(curr_2021.data_2021 + " <-> ");

        curr_2021 = curr_2021.next_2021;

    }

    System.out.println();

}

public static void main(String[] args) {

    NodeDLL_2511532021 head_2021 = new NodeDLL_2511532021(1);

    head_2021.next_2021 = new NodeDLL_2511532021(2);

    head_2021.next_2021.prev_2021 = head_2021;

    head_2021.next_2021.next_2021 = new NodeDLL_2511532021(3);

    head_2021.next_2021.next_2021.prev_2021 = head_2021.next_2021;

        head_2021.next_2021.next_2021.next_2021    =    new
NodeDLL_2511532021(4);

        head_2021.next_2021.next_2021.next_2021.prev_2021    =
head_2021.next_2021.next_2021;

        head_2021.next_2021.next_2021.next_2021.next_2021    =new
NodeDLL_2511532021(5);

        head_2021.next_2021.next_2021.next_2021.next_2021.prev_2021    =
head_2021.next_2021.next_2021.next_2021;

```

```
System.out.print("DLL awal : ");
```

```
printList(head_2021);
```

```
System.out.println("Setelah head dihapus : ");
```

```
head_2021 = delHead_2021(head_2021);
```

```
printList(head_2021);
```

```
System.out.print("setelah node terakhir dihapus: ");
```

```
head_2021 = delLast(head_2021);
```

```
printList(head_2021);
```

```
System.out.print("menghapus node ke 2: ");
```

```
head_2021 = delPost(head_2021,2);
```

```
printList(head_2021);
```

```
}
```

```
}
```

BAB 3 TUGAS

3.5 Program Lagu

```
2 package pekan6_2511532021;
3
4 public class Lagu_2511532021 {
5
6     private String judul_2021;
7     private String penyanyi_2021;
8     private Lagu_2511532021 next_2021;
9     private Lagu_2511532021 prev_2021;
10
11     // konstruktor
12     public Lagu_2511532021(String judul_2021, String penyanyi_2021) {
13         this.judul_2021 = judul_2021;
14         this.penyanyi_2021 = penyanyi_2021;
15         this.next_2021 = null;
16         this.prev_2021 = null;
17     }
18
19     // getter dan setter
20     public String getJudul_2021() { return judul_2021; }
21     public void setJudul_2021(String judul_2021) { this.judul_2021 =
judul_2021; }
22
23     public String getPenyanyi_2021() { return penyanyi_2021; }
24     public void setPenyanyi_2021(String penyanyi_2021) {
this.penyanyi_2021 = penyanyi_2021; }
25
26     public Lagu_2511532021 getNext_2021() { return next_2021; }
27     public void setNext_2021(Lagu_2511532021 next_2021) {
this.next_2021 = next_2021; }
28
29     public Lagu_2511532021 getPrev_2021() { return prev_2021; }
30     public void setPrev_2021(Lagu_2511532021 prev_2021) {
this.prev_2021 = prev_2021; }
31 }
```

Class Lagu_2511532021 adalah representasi node dalam Doubly Linked List untuk menyimpan data lagu, yang memiliki empat field: judul_2021 dan penyanyi_2021 bertipe String untuk menyimpan informasi lagu, serta next_2021 dan prev_2021 bertipe Lagu_2511532021 sebagai pointer ke node berikutnya dan sebelumnya. Konstruktornya menerima dua parameter judul_2021 dan penyanyi_2021 untuk menginisialisasi data lagu, sementara next_2021 dan prev_2021 diset null karena node baru belum terhubung ke node manapun, dan seluruh field diakses melalui getter dan setter yang disediakan.

3.2 Program Musik

```
4 package pekan6_2511532021;
5 import java.util.Scanner;
6
7 public class Musik_2511532021 {
8     private Lagu_2511532021 head_2021;
9     private Lagu_2511532021 tail_2021;
10
11     public Musik_2511532021() {
12         this.head_2021 = null;
13         this.tail_2021 = null;
14     }
15
16     public void tambahLagu_2021(String judul_2021, String
17     penyanyi_2021) {
18         Lagu_2511532021 nodeBaru_2021 = new
19         Lagu_2511532021(judul_2021, penyanyi_2021);
20
21         if (head_2021 == null) {
22             head_2021 = nodeBaru_2021;
23             tail_2021 = nodeBaru_2021;
24         } else {
25             tail_2021.setNext_2021(nodeBaru_2021);
26             nodeBaru_2021.setPrev_2021(tail_2021);
27             tail_2021 = nodeBaru_2021;
28         }
29         System.out.println("Lagu berhasil ditambahkan!");
30     }
31
32     public void hapusLaguAwal_2021() {
33         if (head_2021 == null) {
34             System.out.println("Playlist kosong, tidak ada lagu yang
35             bisa dihapus.");
36         }
37     }
38 }
```

```

34     }
35     System.out.println("Menghapus lagu: " +
head_2021.getJudul_2021());
36     head_2021 = head_2021.getNext_2021();
37
38     if (head_2021 != null) {
39         head_2021.setPrev_2021(null);
40     } else {
41         tail_2021 = null;
42     }
43 }
44
45 public void tampilMaju_2021() {
46     if (head_2021 == null) {
47         System.out.println("Playlist kosong.");
48         return;
49     }
50     Lagu_2511532021 temp_2021 = head_2021;
51     int no_2021 = 1;
52     System.out.println("\n=== Playlist Musik (Maju) ===");
53     while (temp_2021 != null) {
54         System.out.println(no_2021 + ". " +
temp_2021.getJudul_2021() + " - " + temp_2021.getPenyanyi_2021());
55         temp_2021 = temp_2021.getNext_2021();
56         no_2021++;
57     }
58 }
59
60 public void tampilMundur_2021() {
61     if (tail_2021 == null) {
62         System.out.println("Playlist kosong.");
63         return;
64     }
65     Lagu_2511532021 temp_2021 = tail_2021;
66     int no_2021 = 1;
67     System.out.println("\n=== Playlist Musik (Mundur) ===");
68     while (temp_2021 != null) {
69         System.out.println(no_2021 + ". " +
temp_2021.getJudul_2021() + " - " + temp_2021.getPenyanyi_2021());
70         temp_2021 = temp_2021.getPrev_2021();
71         no_2021++;
72     }
73 }
74
75 public void cariLagu_2021(String cariJudul_2021) {
76     if (head_2021 == null) {
77         System.out.println("Playlist kosong.");
78         return;

```

```

79     }
80     Lagu_2511532021 temp_2021 = head_2021;
81     boolean ditemukan_2021 = false;
82     int posisi_2021 = 1;
83
84     while (temp_2021 != null) {
85         if
86         (temp_2021.getJudul_2021().equalsIgnoreCase(cariJudul_2021)) {
87             System.out.println("Lagu ditemukan di urutan ke-" +
88             posisi_2021 + " -> " + temp_2021.getJudul_2021() + " - " +
89             temp_2021.getPenyanyi_2021());
90             ditemukan_2021 = true;
91             break;
92         }
93         temp_2021 = temp_2021.getNext_2021();
94         posisi_2021++;
95     }
96
97     if (!ditemukan_2021) {
98         System.out.println("Lagu dengan judul '" + cariJudul_2021
99         + "' tidak ditemukan.");
100     }
101 }
102
103 public static void main(String[] args) {
104     Scanner sc_2021 = new Scanner(System.in);
105     Musik_2511532021 spotify_2021 = new Musik_2511532021();
106     int pilihan_2021;
107
108     do {
109         System.out.println("\nPlaylist Musik NIM: 2511532021");
110         System.out.println("1. Tambah Lagu");
111         System.out.println("2. Hapus Lagu Pertama");
112         System.out.println("3. Lihat Playlist (Maju)");
113         System.out.println("4. Lihat Playlist (Mundur)");
114         System.out.println("5. Cari Lagu");
115         System.out.println("6. Keluar");
116         System.out.print("Pilihan: ");
117         pilihan_2021 = sc_2021.nextInt();
118         sc_2021.nextLine();
119
120         switch (pilihan_2021) {
121             case 1:
122                 System.out.print("Judul: ");
123                 String judul_2021 = sc_2021.nextLine();
124                 System.out.print("Penyanyi: ");
125                 String penyanyi_2021 = sc_2021.nextLine();

```

```

122         spotify_2021.tambahLagu_2021(judul_2021,
    penyanyi_2021);
123         break;
124     case 2:
125         spotify_2021.hapusLaguAwal_2021();
126         break;
127     case 3:
128         spotify_2021.tampilMaju_2021();
129         break;
130     case 4:
131         spotify_2021.tampilMundur_2021();
132         break;
133     case 5:
134         System.out.print("Cari Judul: ");
135         String cari_2021 = sc_2021.nextLine();
136         spotify_2021.cariLagu_2021(cari_2021);
137         break;
138     case 6:
139         System.out.println("Program selesai.");
140         break;
141     default:
142         System.out.println("Pilihan tidak valid.");
143     }
144     } while (pilihan_2021 != 6);
145     sc_2021.close();
146 }
147 }

```

Class Musik_2511532021 mengimplementasikan playlist musik berbasis Doubly Linked List dengan dua pointer head_2021 dan tail_2021 untuk memudahkan traversal dua arah, serta menyediakan lima operasi utama: tambahLagu_2021 untuk menambah lagu di akhir playlist, hapusLaguAwal_2021 untuk menghapus lagu pertama dengan menggeser head dan memutus pointer prev-nya, tampilMaju_2021 untuk menampilkan playlist dari head ke tail, tampilMundur_2021 untuk menampilkan playlist dari tail ke head menggunakan pointer prev, dan cariLagu_2021 untuk mencari lagu berdasarkan judul secara case-insensitive. Semua operasi tersebut diakses melalui menu interaktif di main yang berjalan dalam loop do-while hingga pengguna memilih opsi keluar.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Kelima file kode tersebut secara keseluruhan mengimplementasikan struktur data Doubly Linked List dalam dua konteks: konteks generik menggunakan NodeDLL, InsertDLL, PenelusuranDLL, dan HapusDLL yang mengoperasikan data bertipe int dengan operasi dasar insert, traversal, dan hapus di awal/akhir/posisi tertentu, serta konteks terapan menggunakan Lagu dan Musik yang mengaplikasikan DLL untuk membangun sistem playlist musik dengan operasi tambah, hapus, tampil maju/mundur, dan pencarian lagu melalui menu interaktif — keduanya mengandalkan prinsip yang sama yaitu setiap node memiliki pointer next dan prev untuk mendukung koneksi dan traversal dua arah.